Lecture 2
Boolean and set theoretic models

Boolean model

Fuzzy sets/
Fuzzy IR

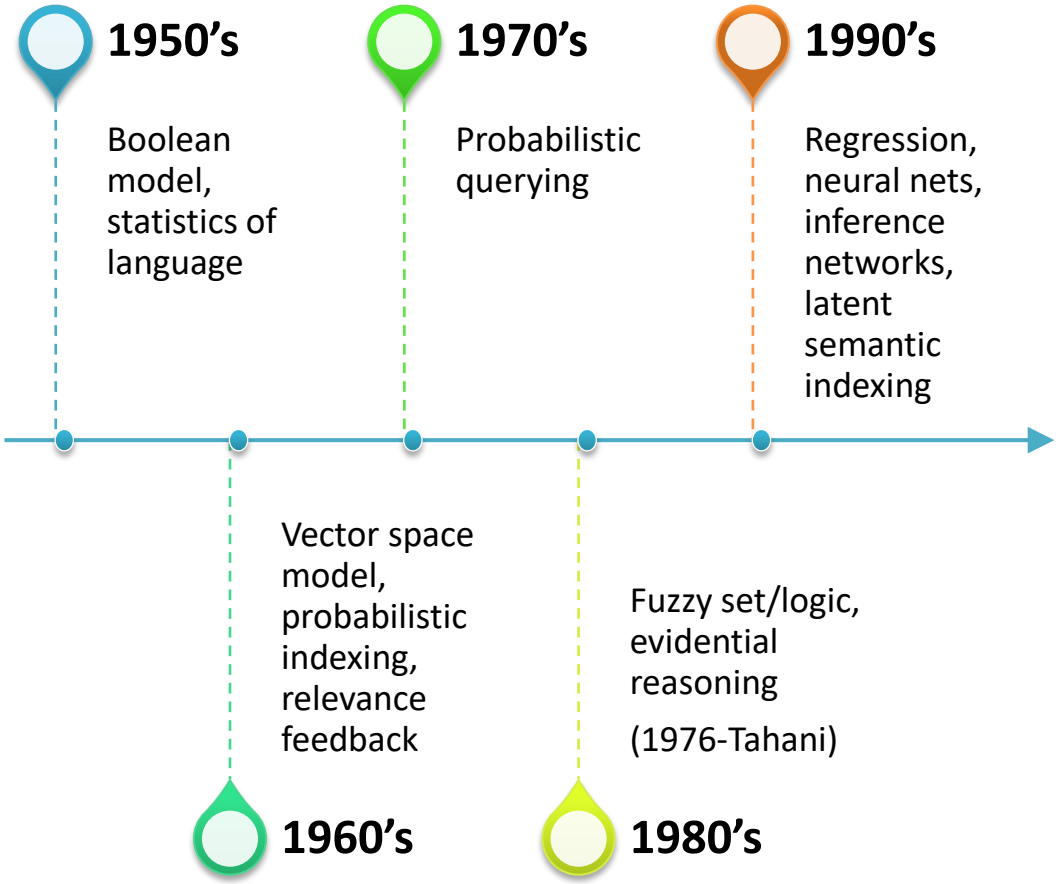
Documents
as sets

Other
extensions

This presentation uses slides (adapted) from presentations of:

- Rada Mihalcea
- Domenic Fletcher

Short historical view of IR



Boolean model

Is (arguably) one of the simplest model.
Primary commercial retrieval tool for 3 decades.
Many searchers still like Boolean queries.

- View each document as a **set** of terms

  Each term is either present or absent

- Queries are Boolean expressions:

  use **and**, **or** and **not** to join query terms

- Output:

  Precise: Document matches condition or not

  No partial matches or ranking.

Boolean models are integrated in search engines

Boolean Search on LinkedIn

professional networking platform

- **NOT searches** programmer NOT manager
- **OR searches** sales OR marketing OR advertising
- **AND searches** accountant AND finance AND CPA

  You don't need to use AND. If your search has two or more terms, you'll automatically see results that include all of them.
- **Quotes " "** Exact phrase match
- **Parenthetical searches** - combine terms using parentheses

  VP NOT (assistant OR SVP)

  find people who have "VP" in their profiles,
  but exclude assistant or SVPs

(https://www.linkedin.com/help/linkedin/answer/a524335/using-boolean-search-on-linkedin?lang=en, accesed: 2025)

Boolean models are integrated in search engines

Boolean Search on LinkedIn

professional networking platform

<u>E.g.</u>:

("Machine Learning Engineer" OR "AI Engineer")
AND Python
NOT Intern

Find people whose title is either "Machine Learning Engineer" or "AI Engineer", who have Python skills, and who are not interns.

Boolean models are integrated in search engines

Westlaw (https://legal.thomsonreuters.com/en/westlaw)

The largest commercial **legal search** service (in terms of the number of paying sub-scribers), with over half a million subscribers performing millions of searches a day over tens of terabytes of text data **(Manning, IIR)**

- Started 1975
- Commercially successful Boolean retrieval

Example query:

Information need: Is a host responsible for drunk guests' behavior?

Query: host! /p (responsib! **or** liab!) /p (intoxicat! **or** drunk!) /p guest

- /p ask for matches in the same paragraph
- The exclamation mark (!) gives a trailing wildcard query : thus responsib! matches all words starting with responsib

Boolean models are integrated in search engines

Westlaw:

online
legal research service
and database

CONNECTORS AND EXPANDERS		
Terms	**Explanation**	**Examples**
&	AND	narcotics & warrant
/s	In same sentence	design /s defect
or	OR	attorney or lawyer or counsel attorney lawyer counsel
+s	Preceding within sentence	attorney +s fee
/p	In same paragraph	hearsay /p utterance
""	Phrase	"res ipsa loquitur"
+p	Preceding within paragraph	exception +p non-liability
%	But not	tax taxation % investment
/n	Within n terms of	personal /3 jurisdiction
!	Root expander	object! retrieves: object, objected, objection, objecting
+n	Preceding within n terms of	capital +3 gain
*	Universal character	withdr*w retrieves: withdraw, withdrew
#	Prefix to turn off plurals and equivalents	#damage retrieves: damage does not retrieve: damages

(https://researchguides.smu.edu.sg/c.php?g=449608&p=3068967 , accesed: 2025)

Boolean Model

Characteristics

- Clean Formalism
- Easy to implement
- Intuitive concept

- More like data retrieval than information retrieval
- *Hard* to translate a text query into a Boolean expression
- All terms are equally weighted
- No ranked output
- No notion for partial matching

Documents as sets

Set based similarity measures

- simple-matching
- overlap
- Dice
- Jaccard
- Tversky index
- cos

- extensions

- maximum values
- relations between them

Documents as sets

Set based similarity measures

- extensions

- $simi_{simple\text{-}matching}(A,B) = |A \cap B|$
- $simi_{overlap}(A,B) = \frac{|A \cap B|}{\min(|A|, |B|)}$
- $simi_{Dice}(A,B) = \frac{2 * |A \cap B|}{|A| + |B|}$
- $simi_{Jaccard}(A,B) = \frac{|A \cap B|}{|A \cup B|}$
- $simi_{Tversky\ index}(A,B) = \frac{|A \cap B|}{|A \cap B| + \propto * |A \backslash B| + \beta * |B \backslash A|}$
- $simi_{cos}(A,B) = \frac{|A \cap B|}{|A|^{1/2} * |B|^{1/2}}$

Properties
- maximum values
- relations between them

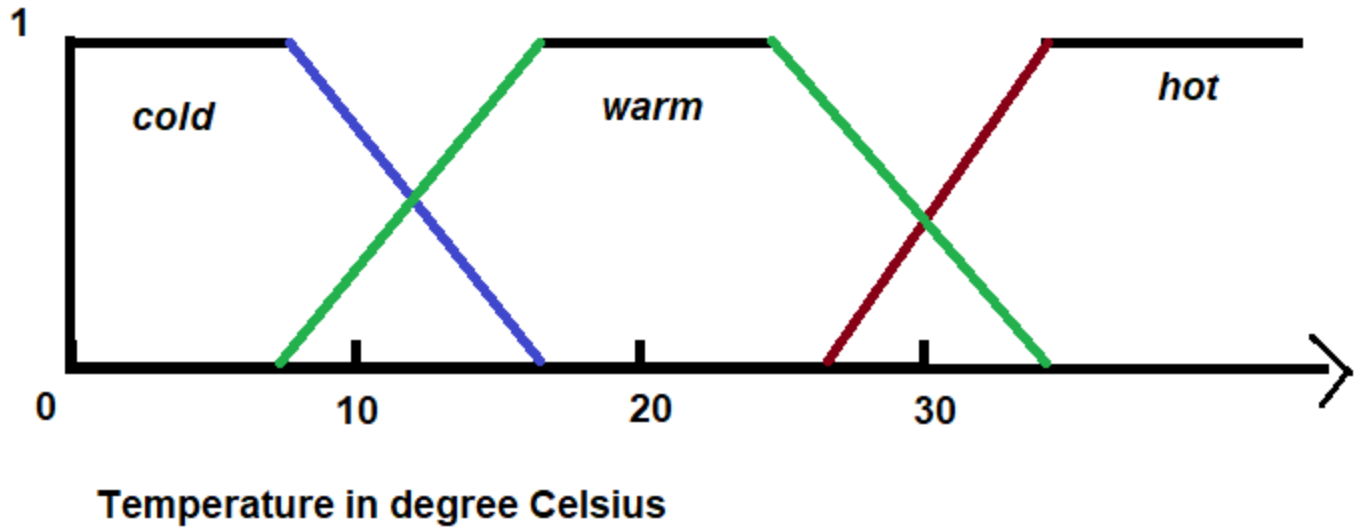
Show that for any 2 non-empty sets A, B:
$simi_{Jaccard}(A,B) = simi_{Dice}(A,B) / (2\text{-}simi_{Dice}(A,B))$

Find an example of sets : A, B, C , D such that:
$simi_{Dice}(A,B) > simi_{Dice}(C,D)$
and $simi_{Jaccard}(A,B) < simi_{Jaccard}(C, D)$

Words and Fuzzy …

Concepts are not sharply defined.

- How do you say cold in numbers?



Fuzzy set model

- The model proposed by Tahani (1976)

  Valiollah Tahani. *A fuzzy model of document retrieval systems*

- The model proposed by Ogawa, Morita, and Kobayashi (1991)

  Yasushi Ogawa, Tetsuya Morita, and Kiyohiko Kobayashi.
  A fuzzy document retrieval system using the keyword connection matrix and a learning method.

Fuzzy Set Theory

a class for query term document collection

A fuzzy subset A of a universe of discourse U

is characterized by a membership function $\mu_A: U \rightarrow [0,1]$ which associates with each element u of U

a number $\mu_A(u)$ in the interval [0,1]

- complement: $\mu_{\overline{A}}(u) = 1 - \mu_A(u)$
- union: $\mu_{A \cup B}(u) = \max(\mu_A(u), \mu_B(u))$
- intersection: $\mu_{A \cap B}(u) = \min(\mu_A(u), \mu_B(u))$

Product ?

- Lacks sensitivity for min & max: min(0.2, 0.8) = 0.2
  min(0.2, 0.3) = 0.2

Fuzzy Logic Model. Example

User query: "*Find a cheap hotel in Cluj-Napoca with good reviews*."

Query: *cheap* AND *in_Cluj-Napoca* AND *good_reviews*

Membership scores:

Hotel	**cheap**	**in_Cluj-Napoca**	**good_reviews**
H1	0.9	1.0	0.7
H2	0.6	1.0	0.9
H3	1.0	1.0	0.8
H4	1.0	0.0	1.0

Using fuzzy AND (minimum):

H1 = min(0.9, 1.0, 0.7) = 0.7
H2 = min(0.6, 1.0, 0.9) = 0.6
H3 = min(1.0, 1.0, 0.8) = 0.8
H4 = min(1.0, 0.0, 1.0) = 0.0

Fuzzy Logic Model. Example

User query: "*Find a cheap hotel in Cluj-Napoca with good reviews*."

Query: *cheap* AND *in_Cluj-Napoca* AND *good_reviews*

fuzzy concept "cheap hotel"

Assume a booking site contains these prices: 30, 40, 50, 55, 60, 70, 80, 90, 150, 200, 250

Lowest price: 30

Median price: 80

Mean price: 140 Upper threshold for cheap: 150

Membership scores:

Hotel	**Price**	***Cheap* membership**
H1	50	0.9
H2	80	0.6
H3	40	1.0
H4	30	1.0

Or, we can define a function. e.g.:

$$\mu_{cheap}(p) = \begin{cases} 1 & p \leq 50 \\ \frac{150-p}{100} & 50 < p < 150 \\ 0 & p \geq 150 \end{cases}$$

Fuzzy Logic Model

How to assign membership degree values for terms in a document:

- Based on **Term Frequency**: tf /max(tf)
- Based on **TF-IDF** weights: tf-idf / max(tf-idf)
- **Experts** may assign membership values
- Define membership **function** for vague concepts

  e.g.: hotel prices vs cheap hotels
- Machine learning (**ML**)
- Memberships can be derived from **term similarity**

e.g.:
A document D not containing “car”,
but containing "vehicle" partially satisfies the query term "car".
similarity(car, vehicle) = 0.91
=> μ_D(car) = 0.91

Fuzzy Logic Model in IR (Tahani)

The Fuzzy Logic Model (FLM), proposed by Tahani (1976):

- A document is represented by the set of weighted index terms
  e.g. **d1** = (***and*** <jazz,0.4> <sax,0.3> <Coltrane,0.5>)
- Query **qj** - boolean (based on terms **tk**)
- The degree to which document **di** satisfies **qj**

μ (di,tk) = $\boldsymbol{W}_{ki}$

μ (di,(***not*** qj)) = 1 – μ (di,qj)

μ(di,(***and*** q1 .. qn)) = ***min*** { μ(d1,q1),.., μ(di,qn) }

μ (di,(***or*** q1 .. qn)) = ***max*** { μ(d1,q1),.., μ(di,qn) }

Closed word assumption:
the absence of a term t in the representation of d is noted with: <t,0>

Based on: set of words model
boolean query

Fuzzy Logic Model in IR (Ogawa)

Based on: set of words model
boolean query

Model:

- a query term: a fuzzy set
- a document: degree of membership in this set

matching ← approximation

membership function

- Associate membership function with the elements of the class
- 0: no membership in the set
- 1: full membership
- 0~1: marginal elements of the set

Fuzzy Logic Model (Ogawa)

- Uses a **keyword connection matrix** (thesaurus)

  normalized correlation factor between terms

  $c_{i,l}$: (k_i , k_l)

$$c_{i,l} = \frac{n_{i,l}}{n_i + n_l - n_{i,l}}$$ where
- n_i is # of documents containing term k_i
- n_l is # of documents containing term k_l
- $n_{i,l}$ is # of documents containing k_i and k_l

- a document d_j has a degree of membership $\mu_{i,j}$ to the fuzzy set associated to a term k_i :

$$\mu_{i,j} = 1 - \prod_{k_l \in d_j} (1 - c_{i,l})$$

Fuzzy Logic Model (Ogawa)

- an index term k_a has an associated fuzzy set D_a

  $d_j \in D_a$: has a degree of membership

  a document d_j has a degree of membership $\mu_{i,j}$ to the fuzzy set associated to index term k_i :

$$\mu_{i,j} = 1 - \prod_{k_l \in d_j} (1 - c_{i,l})$$

- The degree of membership in a fuzzy set:
  computed based on an algebraic product
  (instead of min/max function)
  (*more smoothly*)
- Complex Query - use the disjunctive normal form

Fuzzy Logic Model (Ogawa). Example

Example:

Determine the similarity between Q and D1, D2, D3

Q: gold AND silver AND truck
D1: "Shipment of gold damaged in a fire"
D2: "Delivery of silver arrived in a silver truck"
D3: "Shipment of gold arrived in a truck"

Use the next dimensions (8 selected terms)

arrived, damaged, delivery, fire, gold, silver, shipment, truck

Fuzzy Logic Model (Ogawa)

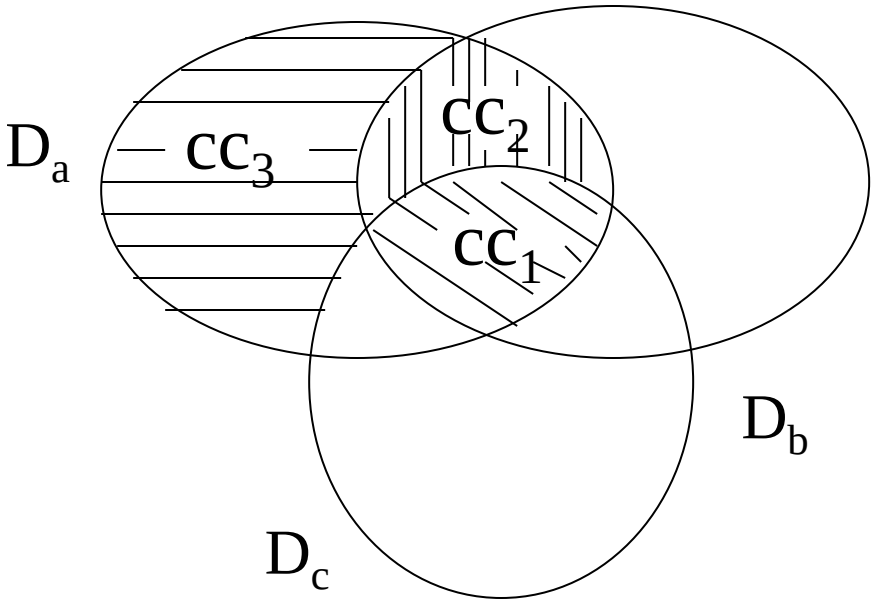
Having a boolean **Query**

use the **disjunctive normal form** of the terms

e.g.:

$$\boldsymbol{q} = a \wedge (b \vee \neg c)$$

$$= cc_1 + cc_2 + cc_3$$



For query ***q***, document ***j*** :

$$\mu_{q,j} = \mu_{cc1+cc2+cc3,j}$$

$$= 1 - \prod_{i=1}^{3} (1 - \mu_{cc_i,j})$$

$$= 1 - (1 - \mu_{a,j}\mu_{b,j}\mu_{c,j}) \times (1 - \mu_{a,j}\mu_{b,j}(1 - \mu_{c,j})) \times (1 - \mu_{a,j}(1 - \mu_{b,j})(1 - \mu_{c,j}))$$

Extended Boolean Model

- introduced by Salton, Fox, and Wu, 1983
  "Extended boolean information retrieval"

Document :
terms: k1, k2, … , kn
weights x1, x2, … , xn

e.g.: for q = kx ∧ ky

use weights associated with kx and ky

- Boolean model:
  weights equivalent to: 0 – 1 values

- Extended model:
  weights : real values, normalized

Produce rankings !

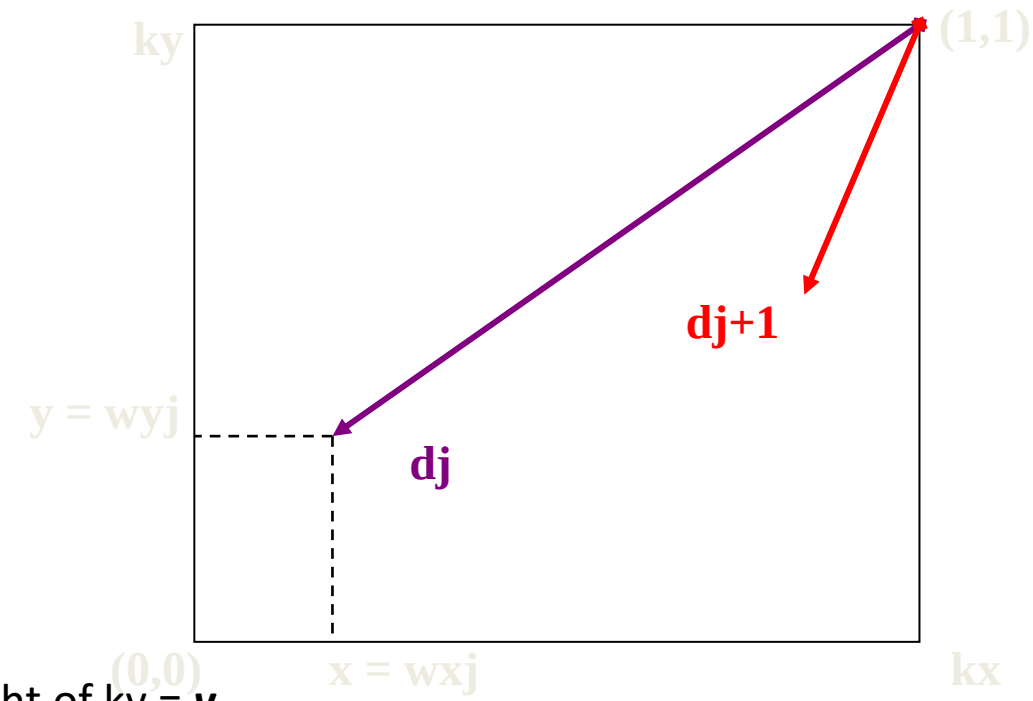
Extended Boolean Model. The Idea

AND

e.g.:

$$q_{and} = \text{kx} \wedge \text{ky}$$

We want a document to be as close as possible to (1,1)



Assume: In document ***d***,
weight of kx is ***x*** *and* weight of ky = ***y***

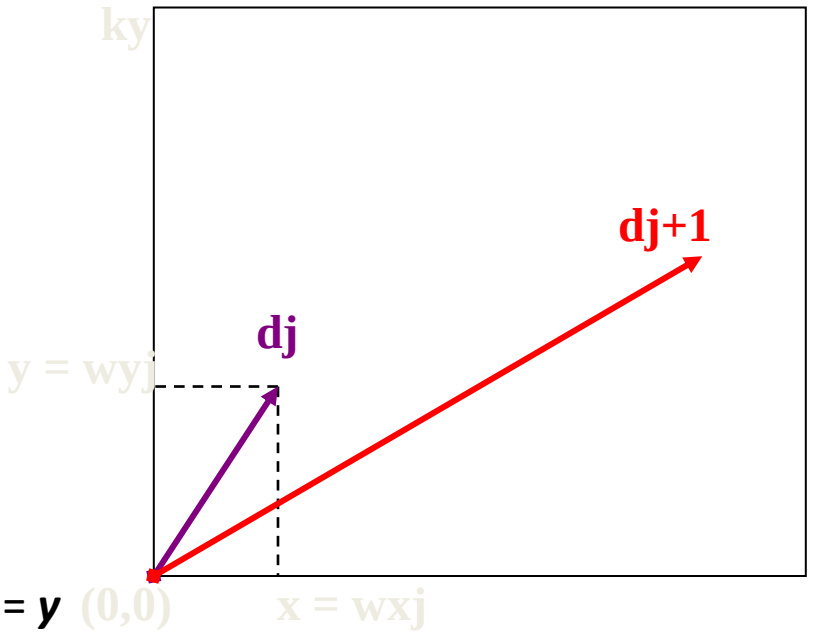
$$sim(q_{and}, d) = 1 - \sqrt{\frac{(1-x)^2 + (1-y)^2}{2}}$$

Extended Boolean Model. The Idea

OR

e.g.:

$$qor = kx \vee ky$$



We want a document to be as far as possible from (0,0)

Assume: In document ***d***,
weight of kx is ***x*** *and* weight of ky = ***y***

$$sim(q_{or}, d) = \sqrt{\frac{x^2 + y^2}{2}}$$

Extended Boolean Model

m terms and p-norm

- $q_{or}=k_1 \vee^p k_2 \vee^p \ldots \vee^p k_m$

$$sim(q_{or}, d_j) = \left(\frac{x_1^p + x_2^p + \ldots + x_m^p}{m}\right)^{1/p}$$

- $q_{and}=k_1 \wedge^p k_2 \wedge^p \ldots \wedge^p k_m$

$$sim(q_{and}, d_j) = 1 - \left(\frac{(1-x_1)^p + (1-x_2)^p + \ldots + (1-x_m)^p}{m}\right)^{1/p}$$

? $q=(k_1 \wedge k_2) \vee k_3$

? Properties
- when p=1 ;
- when p= ∞

Extended Boolean Model

Characteristics

- Works with boolean queries
- Produces rankings
- Allows partial matching

- Use weighted query terms and p-norm similarity functions
- Parameter p= ∞ --> fuzzy logic model

  p=2 --> vector space model (?)

Text retrieval



source: https://trec.nist.gov/ , accessed: 2020